LABORATORIO REALIZADO

LABORATORIO 03

Desarrollo de Aplicaciones Web

Sección C24 D

Jesus Castillo Sumire

API REST para administración de productos y ventas

1. Objetivo y alcance

Implementé una API REST con Spring Boot para practicar persistencia con JPA/Hibernate y organizar la lógica en capas. El trabajo incluye el CRUD de productos solicitado y amplía el ejercicio con un módulo de ventas formado por categorías, productos, clientes, empleados, ventas y detalles de venta.

La aplicación valida los datos recibidos, conserva las relaciones entre entidades y centraliza las respuestas de error. El servicio de ventas calcula subtotales y total, descuenta el inventario al registrar una venta y repone el stock si se actualiza o elimina.

2. Organización y flujo

Capa	Responsabilidad
Controller	Recibe HTTP, valida el cuerpo y devuelve recursos/códigos de estado REST.
Service	Aplica reglas del negocio, busca relaciones y administra transacciones.
Repository	Accede a MySQL mediante Spring Data JPA.
Model / DTO	Representa las tablas y valida los datos de entrada.

Flujo general: solicitud HTTP → Controller → Service → Repository/JPA → MySQL; la respuesta vuelve al cliente como JSON.

3. Entidades y relaciones

Categoria 1:N Producto; Cliente 1:N Venta; Empleado 1:N Venta; Venta 1:N DetalleVenta; Producto 1:N DetalleVenta. Las claves foráneas impiden crear registros relacionados con identificadores inexistentes. Cada detalle conserva el precio unitario usado en la venta.

Herramientas: Java 17, Spring Boot, Spring Web MVC, Spring Data JPA, Hibernate, Gradle, MySQL de XAMPP, Visual Studio Code y Postman.

4. Código principal explicado

Modelo Producto — define la entidad persistida y sus restricciones. La categoría obligatoria representa la relación muchos-a-uno.

```
@Entity
@Table(name = "productos")
public class Producto {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank @Size(max = 120)
    @Column(nullable = false, length = 120)
    private String nombre;

    @NotNull @DecimalMin("0.01")
    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal precio;

    @NotNull @Min(0)
    @Column(nullable = false)
    private Integer stock;

    @NotNull
    @ManyToOne(fetch = FetchType.EAGER, optional = false)
    @JoinColumn(name = "categoria_id", nullable = false)
    private Categoria categoria;
}
```

Función: `@Entity` y `@Table` conectan la clase con la tabla; `@Id` genera la clave primaria. Las validaciones evitan nombres vacíos, precios no positivos y stock negativo. `@ManyToOne` crea la referencia a la categoría.

Repositorio — proporciona las operaciones CRUD básicas sin escribir SQL repetitivo.

```
public interface ProductoRepository extends JpaRepository<Producto, Long> {}
```

Función: Spring Data genera métodos como `findAll`, `findById`, `save` y `delete` usando el tipo de entidad y su identificador.

DTO de entrada — limita y valida lo que se puede enviar al crear o actualizar un producto.

```
public record ProductoRequest(
    @NotBlank @Size(max=120) String nombre,
    @NotNull @DecimalMin("0.01") BigDecimal precio,
    @NotNull @Min(0) Integer stock,
    @NotNull Long categoriaId) {}
```

Función: El DTO separa el formato de entrada de la entidad y deja que Bean Validation detecte valores inválidos antes de ejecutar la lógica de negocio.

5. Lógica de negocio y API

Servicio de productos — centraliza el CRUD y verifica que la categoría exista.

```
@Service
public class ProductoService {
    private final ProductoRepository repository;
    private final CategoriaRepository categoriaRepository;

    public List<Producto> listar() { return repository.findAll(); }
    public Producto obtener(Long id) {
        return repository.findById(id).orElseThrow(
            () -> new EntityNotFoundException("Producto " + id + " no encontrado"));
    }
    public Producto crear(ProductoRequest r) {
        return repository.save(new Producto(r.nombre(), r.precio(), r.stock(),
                                            categoria(r.categoriaId())));
    }
    @Transactional public Producto actualizar(Long id, ProductoRequest r) {
        Producto p = obtener(id);
        p.setNombre(r.nombre()); p.setPrecio(r.precio()); p.setStock(r.stock());
        p.setCategoria(categoria(r.categoriaId()));
        return p;
    }
    public void eliminar(Long id) { repository.delete(obtener(id)); }
}
```

Función: El servicio reúne las reglas del recurso. `obtener` produce 404 cuando el ID no existe; `actualizar` carga y modifica la entidad dentro de una transacción; `crear` resuelve la categoría antes de guardar.

Controlador — publica las rutas REST y asigna 201 a las altas y 204 a las eliminaciones.

```
@RestController
@RequestMapping("/api/productos")
public class ProductoController {
    private final ProductoService service;
    public ProductoController(ProductoService service) { this.service = service; }
    @GetMapping public List<Producto> listar() { return service.listar(); }
    @GetMapping("/{id}") public Producto obtener(@PathVariable Long id) {
        return service.obtener(id);
    }
    @PostMapping @ResponseStatus(HttpStatus.CREATED)
    public Producto crear(@Valid @RequestBody ProductoRequest r) {
        return service.crear(r);
    }
    @PutMapping("/{id}") public Producto actualizar(@PathVariable Long id,
            @Valid @RequestBody ProductoRequest r) { return service.actualizar(id, r); }
    @DeleteMapping("/{id}") @ResponseStatus(HttpStatus.NO_CONTENT)
    public void eliminar(@PathVariable Long id) { service.eliminar(id); }
}
```

Función: Las anotaciones enlazan los verbos HTTP con cada método. `@Valid` activa las reglas del DTO y `@RequestBody` convierte JSON a objeto Java.

6. Relaciones, ventas y persistencia

Venta — vincula cliente y empleado con sus detalles. La colección usa cascada para persistir detalles junto con la venta.

```
@Entity
@Table(name = "ventas")
public class Venta {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false) private LocalDateTime fecha;
    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private EstadoVenta estado = EstadoVenta.REGISTRADA;
    @ManyToOne(optional = false) @JoinColumn(name = "cliente_id", nullable = false)
    private Cliente cliente;
    @ManyToOne(optional = false) @JoinColumn(name = "empleado_id", nullable = false)
    private Empleado empleado;
    @OneToMany(mappedBy = "venta", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<DetalleVenta> detalles = new ArrayList<>();
    public BigDecimal getTotal() {
        return detalles.stream().map(DetalleVenta::getSubtotal)
            .reduce(BigDecimal.ZERO, BigDecimal::add);
    }
}
```

Función: La entidad expresa las claves foráneas y la relación uno-a-muchos. El total se calcula sumando subtotales para evitar guardar un total desactualizado.

VentaService — controla inventario junto con la operación de venta.

```
@Transactional public Venta crear(VentaRequest r) {
    Venta venta = new Venta(cliente(r.clienteId()), empleado(r.empleadoId()), estado(r.estado()));
    agregarDetalles(venta, r.detalles());
    return ventas.save(venta);
}

private void agregarDetalles(Venta venta, List<DetalleVentaRequest> solicitudes) {
    for (DetalleVentaRequest solicitud : solicitudes) {
        Producto producto = productos.findById(solicitud.productoId())
            .orElseThrow(() -> new EntityNotFoundException("Producto no encontrado"));
        if (producto.getStock() < solicitud.cantidad())
            throw new StockInsuficienteException("Stock insuficiente");
        producto.setStock(producto.getStock() - solicitud.cantidad());
        venta.getDetalles().add(new DetalleVenta(venta, producto,
            solicitud.cantidad(), producto.getPrecio()));
    }
}
```

Función: La transacción hace que el registro de la venta y la reducción de inventario se confirmen juntos. Si no hay stock, se cancela toda la operación.

7. Base de datos y errores

La aplicación usa MySQL en `localhost:3306`, base `laboratorio_api`. `createDatabaseIfNotExist=true` solicita crear la base cuando no exista y el usuario tenga permisos. Hibernate está configurado con `ddl-auto=update` para sincronizar tablas a partir de las entidades. También se incluye el script SQL `database/laboratorio_api.sql`.

```
spring.datasource.url=jdbc:mysql://localhost:3306/laboratorio_api?createDatabaseIfNotExist=true&useSSL=false&ser
spring.datasource.username=root
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
server.port=8080
```

Función: Estas propiedades indican conexión, credenciales locales, actualización del esquema y puerto HTTP. En otro equipo se deben ajustar usuario/contraseña de MySQL según la instalación.

El manejador global traduce excepciones comunes a respuestas HTTP: registro inexistente 404, errores de validación 400, y conflicto de integridad/stock 409. El CRUD de productos usa `/api/productos`; los demás recursos siguen `/api/categorias`, `/api/clientes`, `/api/empleados`, `/api/ventas` y `/api/ventas/{ventaId}/detalles`.

8. Postman y verificación

La colección `postman/Semana03-Modulo-Ventas.postman_collection.json` contiene una secuencia CRUD de productos (listar, crear, consultar, actualizar, validar precio inválido y eliminar) y una prueba de creación/consulta de venta. Para ejecutarla: iniciar MySQL en XAMPP, iniciar el proyecto en el puerto 8080, importar el archivo JSON en Postman y ejecutar la colección.

Estado comprobado en esta entrega: la compilación de Java terminó correctamente. No se pudo iniciar el servidor MySQL de XAMPP en esta sesión porque el archivo de datos `ibdata1` quedó bloqueado/no escribible; por eso no se ejecutaron las solicitudes Postman y no se incluyen capturas de respuestas que no hayan ocurrido. La colección queda preparada para correrla en el equipo cuando MySQL esté disponible.

Respuestas esperadas al ejecutar con la base activa: GET 200; POST válido 201; PUT 200; DELETE 204; entrada inválida 400; ID inexistente 404; falta de stock o conflicto de integridad 409. Estos son los estados definidos por el código, no resultados de una corrida de Postman en esta sesión.

9. Conclusiones

1. Con este laboratorio reforcé cómo se organiza una API REST por capas: el controlador atiende las rutas, el servicio aplica las reglas y el repositorio se encarga del acceso a la base de datos. También comprobé que las entidades JPA permiten representar las relaciones entre los datos sin escribir todas las consultas manualmente.

2. En el módulo de ventas entendí que registrar una operación requiere cuidar la consistencia: validar cliente, empleado y productos, calcular el total y actualizar el stock dentro de una transacción. La compilación confirmó que el código Java está integrado; queda ejecutar la colección de Postman cuando MySQL de XAMPP pueda iniciar.

Archivos principales del laboratorio

Proyecto: `Semana03/modulo-ventas-api`
Esquema: `database/laboratorio_api.sql`
Colección: `postman/Semana03-Modulo-Ventas.postman_collection.json`
Ejecución: `.\gradlew.bat bootRun`